Week 6: Building a WireGuard VPN Between Kali and pfSense

The short version

I built a WireGuard VPN tunnel between my Kali machine (the remote client) and my pfSense firewall (the VPN server), then used Wireshark to prove the tunnel really hides the traffic. It took several tries because of small mistakes, and each one taught me how to *verify* that a VPN works instead of assuming it does.

What a VPN is, in plain words

A VPN is a private, encrypted tunnel between two machines across a network they do not trust. Anyone watching the network between them sees scrambled data, not the real messages.

- **Remote-access VPN (what I built):** one device (Kali, playing a laptop on the road) connects into a network (pfSense, playing the office).
- Many people think a VPN is only for hiding your location. That is one use. The security use is protecting data in transit.

Why WireGuard

WireGuard is a modern VPN protocol. It is small (a few thousand lines of code, so easier to audit), fast, and uses fixed modern cryptography instead of letting you pick weak options. IPsec and OpenVPN are older alternatives: more flexible, but harder to configure and audit.

The lab

Machine	Role	Address
Kali	VPN client	eth1 192.168.58.10, tunnel wg0 10.10.10.2
pfSense	VPN server	LAN em0 192.168.58.1, tunnel 10.10.10.1, UDP port 51820

Tunnel network: 10.10.10.0/24.

How WireGuard works (the key ideas)

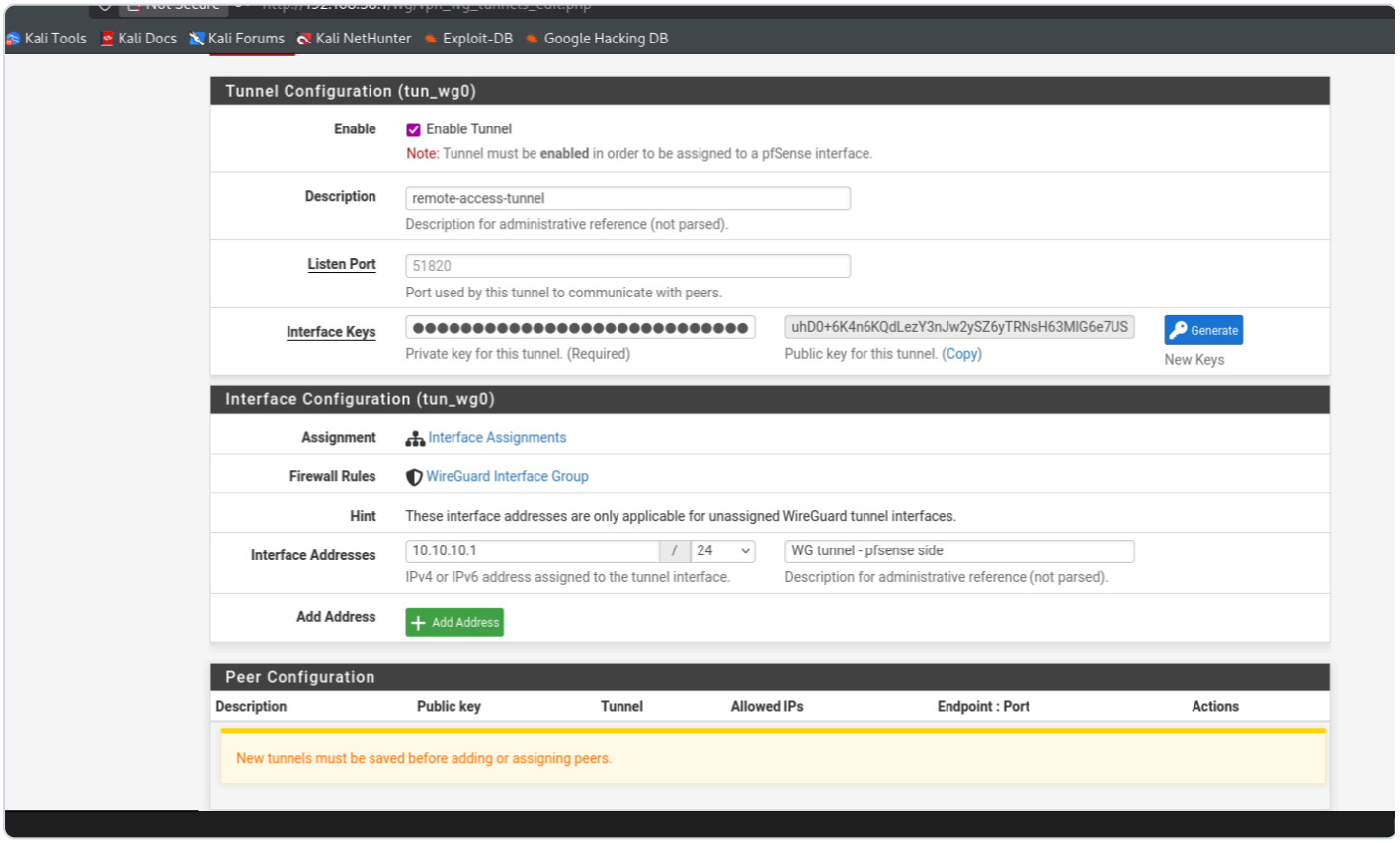
- Each side has a **key pair**: a private key (never shared) and a public key (given to the other side).
- Each side needs the **other side's public key**. If the key one end holds does not match the real key of the other end, they cannot trust each other and nothing happens.
- Asymmetric cryptography (public and private keys) is used once, during the **handshake**, to agree on a shared secret. Then fast symmetric encryption (ChaCha20-Poly1305) protects the actual traffic. This is a hybrid cryptosystem.

- `AllowedIPs` says which addresses may be sent through the tunnel. It acts like a routing table and a firewall rule for that peer.
- `PersistentKeepalive` sends a small packet every 25 seconds so the tunnel stays open.

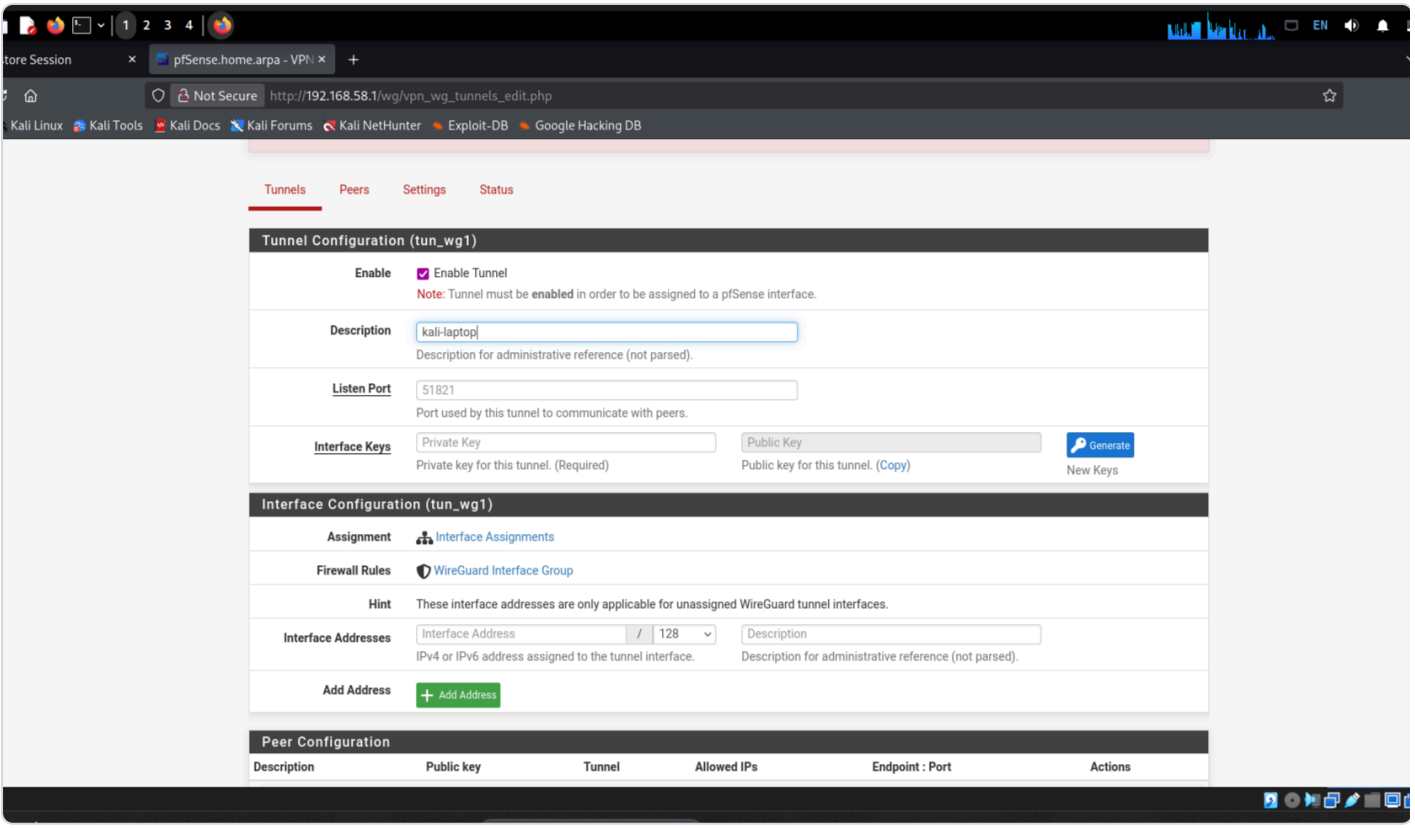
> Security note: I never publish private keys. The screenshots below show only public keys, which are safe to share. I left out any screenshot that showed a private key.

What I built

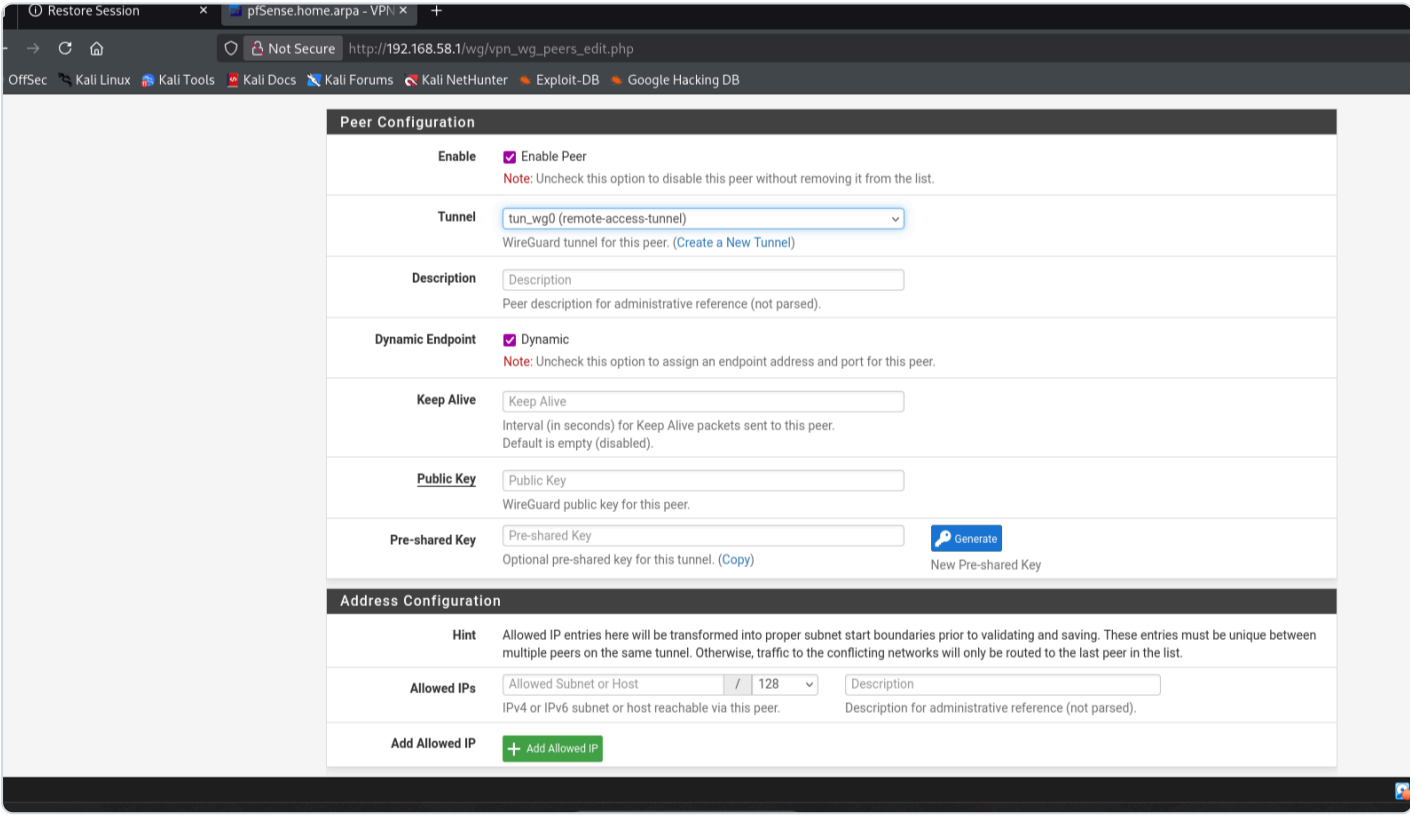
On pfSense: installed the WireGuard package, created tunnel `tun_wg0` (port 51820, address 10.10.10.1/24), and added Kali as a peer.



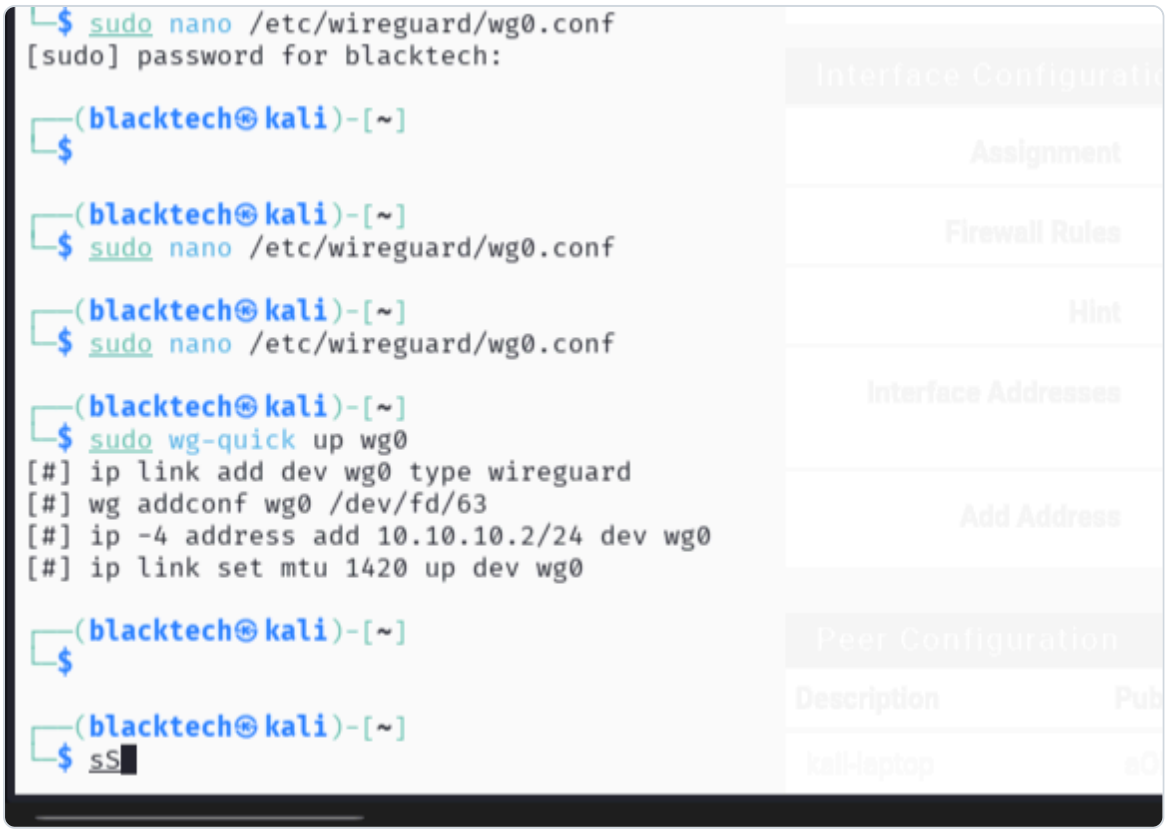
I also created a second tunnel (`tun_wg1`, "kali-laptop") by mistake. It is still on my to-do list to delete it.



The peer form (Kali's public key and allowed IPs go here):



On Kali: installed WireGuard, generated a key pair, and wrote `/etc/wireguard/wg0.conf` with an `[Interface]` and a `[Peer]` section (endpoint is pfSense at 192.168.58.1:51820). Brought it up with `sudo wg-quick up wg0`.



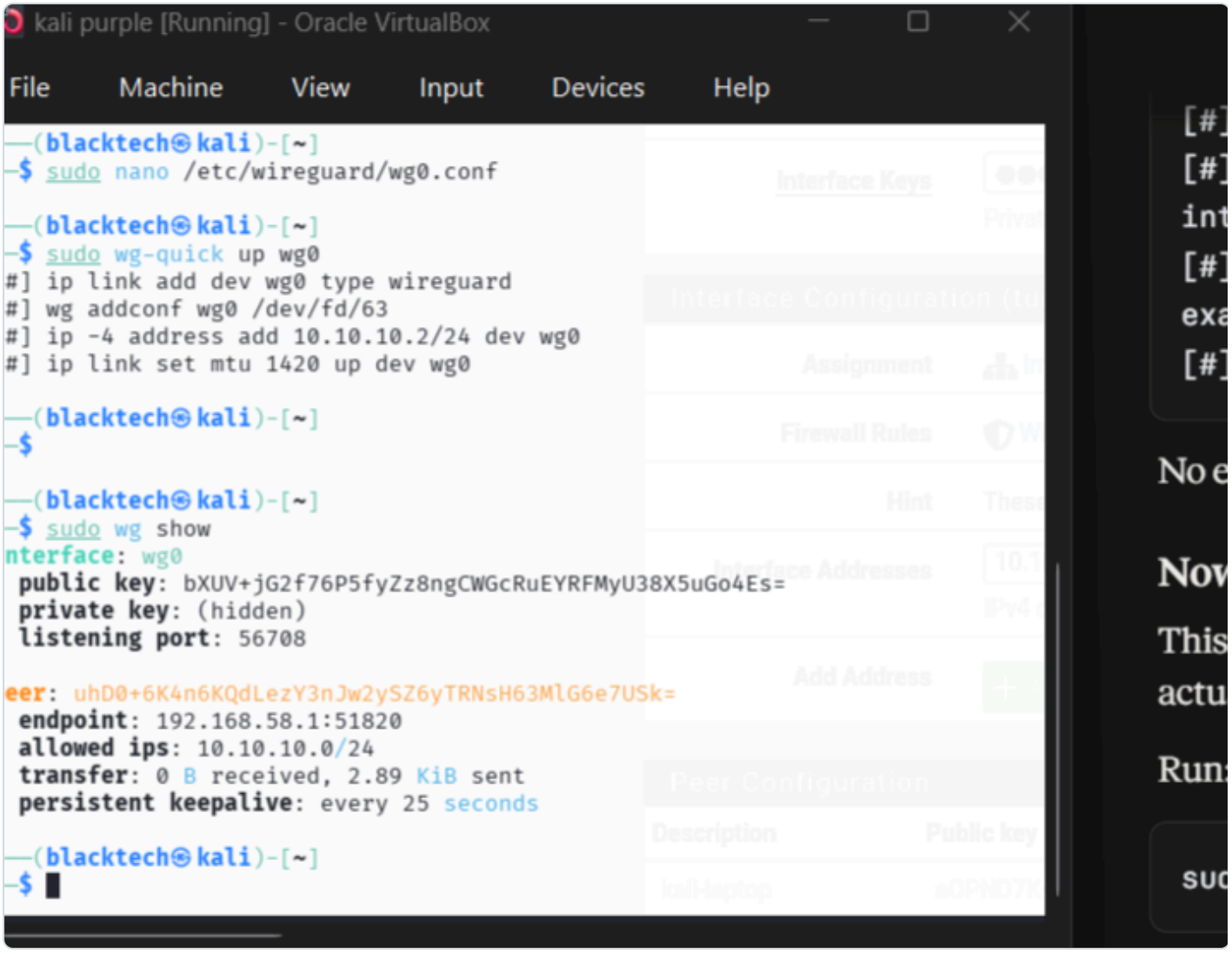
What went wrong, and how I fixed it

Mistake 1: typos in the config file

WireGuard keywords must be spelled exactly. I typed `Adress`, `[peer]`, `Privatekey` and `Publickey`. The tool silently refused or misread them. Fix: use the exact names (`Address`, `[Peer]`, `PrivateKey`, `PublicKey`).

Mistake 2: the handshake never completed

`wg show` had no "latest handshake" line and **0 B received**. I only ever sent data (the keepalives) and never heard back:



This happened more than once (Oct 4, Oct 10, Oct 17):

```
  listening port: 59919

peer: uhD0+6K4n6KQdLezY3nJw2ySZ6yTRNsH63MlG6e7USk=
  endpoint: 192.168.58.1:51820
  allowed ips: 10.10.10.0/24
  transfer: 0 B received, 2.75 KiB sent
  persistent keepalive: every 25 seconds

┌──(blacktech㉿kali)-[~]
└─$
```

```
Session  Actions  Edit  View  Help
┌──(blacktech㉿kali)-[~]
└─$ sudo wg-quick up wg0
[sudo] password for blacktech:
[#] ip link add dev wg0 type wireguard
[#] wg addconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0

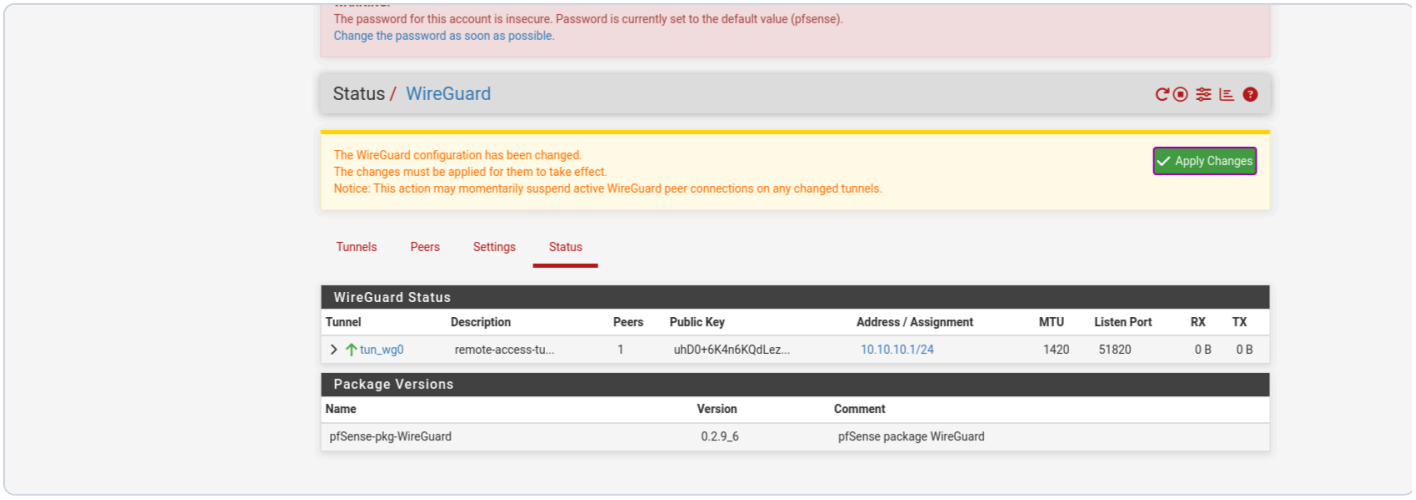
┌──(blacktech㉿kali)-[~]
└─$ sudo wg show
interface: wg0
  public key: bXUV+jG2f76P5fyZz8ngCWGcRuEYRFMyU38X5uGo4Es=
  private key: (hidden)
  listening port: 53989

peer: uhD0+6K4n6KQdLezY3nJw2ySZ6yTRNsH63MlG6e7USk=
  endpoint: 192.168.58.1:51820
  allowed ips: 10.10.10.0/24
  transfer: 0 B received, 1.16 KiB sent
  persistent keepalive: every 25 seconds

┌──(blacktech㉿kali)-[~]
└─$
```

Two causes:

1. The public key I pasted into pfSense for Kali did not match Kali's real public key.
2. pfSense showed a yellow **Apply Changes** banner that I had not clicked, so the saved settings were not active.



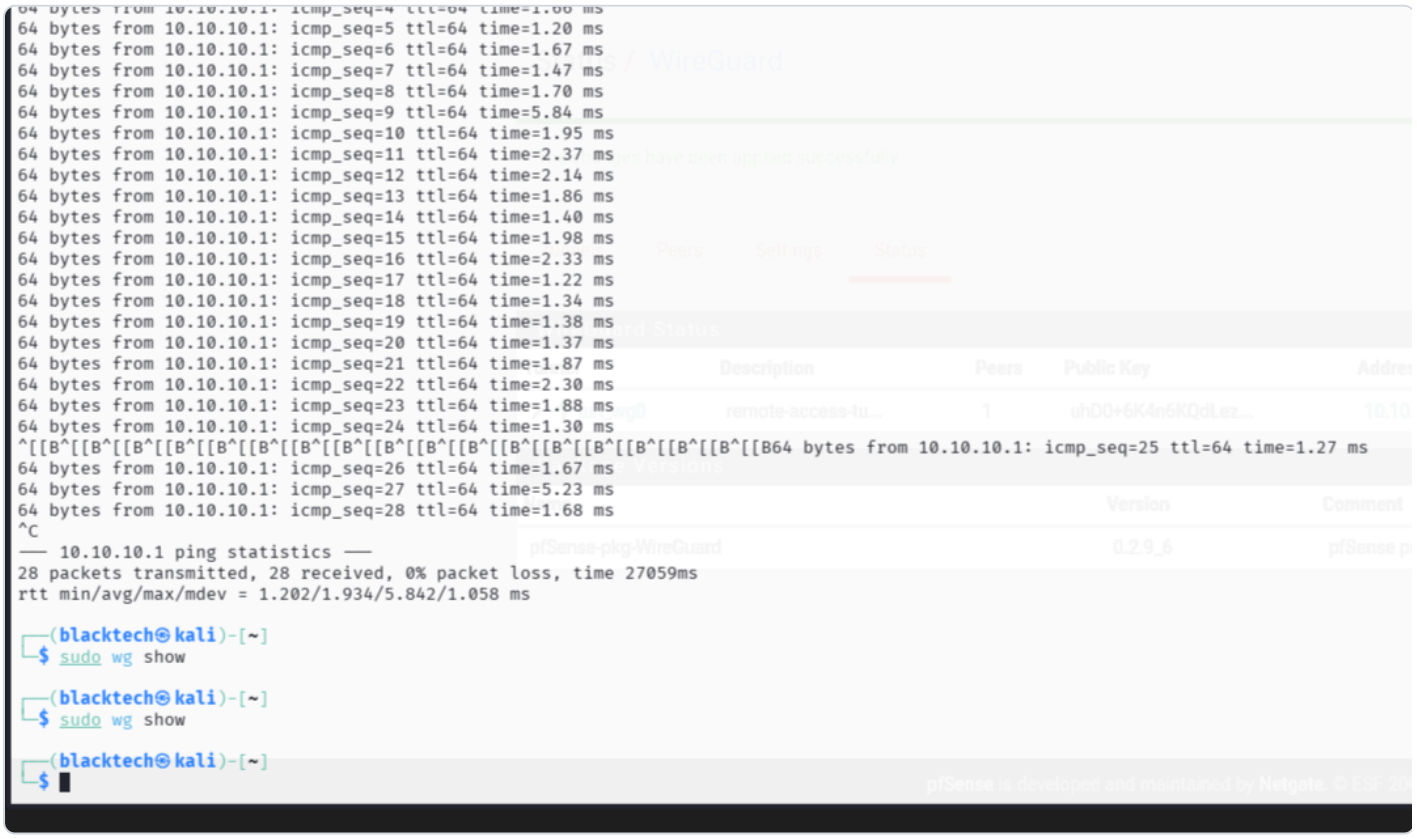
Fix: read the real key with `sudo wg show wg0 public-key`, paste it into the pfSense peer, click Save, then click **Apply Changes**.

On Oct 17 the ping failed (100% loss) until the handshake finally appeared, with data now received:

```
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0

┌──(blacktech㉿kali)-[~]
└─$ ping -c 4 10.10.10.1
PING 10.10.10.1 (10.10.10.1) 56(84) bytes of data.
^[[B^[[B^[[B
```

On Oct 10 a ping to 10.10.10.1 reported 28 of 28 replies:



Yet `wg show` on the same day still said 0 B received (screenshot 08 above). A ping succeeding does not prove the tunnel carried it. The proper test is to ask Kali which way it will send the packet:

```
ip route get 10.10.10.1
```

It must say `dev wg0`. If it says `eth1`, the traffic is not using the VPN, even though the ping works.

Lesson: a success message does not prove the thing you think it proves. Check the path, not only the result.

Proof: the tunnel works

1. `sudo wg show` shows a **latest handshake** and non-zero received data:

```
┌──(blacktech㉿kali)-[~]
└─$ ping -c 4 10,10.10.1
ping: 10,10.10.1: Name or service not known

┌──(blacktech㉿kali)-[~]
└─$ sudo wg-quick up wg0
[sudo] password for blacktech:
[#] ip link add dev wg0 type wireguard
[#] wg addconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.10.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0

┌──(blacktech㉿kali)-[~]
└─$ sudo wg show
interface: wg0
  public key: bXUV+jG2f76P5fyZz8ngCWGcRuEYRFMyU38X5uGo4Es=
  private key: (hidden)
  listening port: 53208

peer: uhD0+6K4n6KQdLezY3nJw2ySZ6yTRNsH63MlG6e7USk=
  endpoint: 192.168.58.1:51820
  allowed ips: 10.10.10.0/24
  latest handshake: 17 seconds ago
  transfer: 124 B received, 180 B sent
  persistent keepalive: every 25 seconds

┌──(blacktech㉿kali)-[~]
└─$ sS
```

1. The `wg0` interface exists with address 10.10.10.2/24:

```
  transfer: 2.04 KiB received, 1.41 KiB sent
  persistent keepalive: every 25 seconds

┌──(blacktech㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:40:a4:f4 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:b4:f2:2c brd ff:ff:ff:ff:ff:ff
    inet 192.168.58.10/24 brd 192.168.58.255 scope global noprefixroute eth1
       valid_lft forever preferred_lft forever
    inet6 fe80::896c:94ce:47cc:33b6/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
4: wg0: <POINTOPOINT,NOARP,UP,LOWER_UP> mtu 1420 qdisc noqueue state UNKNOWN group default qlen 1000
    link/none
    inet 10.10.10.2/24 scope global wg0
       valid_lft forever preferred_lft forever

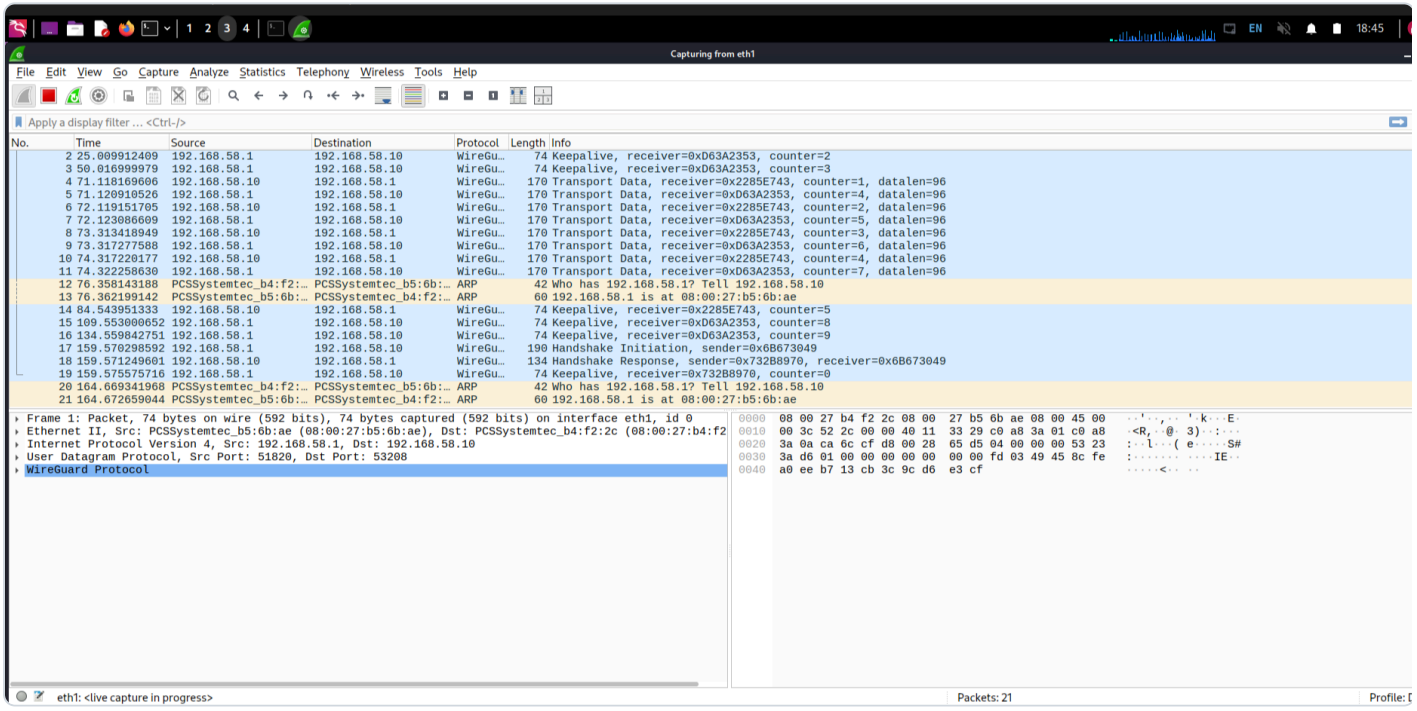
┌──(blacktech㉿kali)-[~]
└─$ sS
```

1. `ip route get 10.10.10.1` shows `dev wg0`. *(Screenshot still to add.)*
2. `ping -c 4 10.10.10.1` returns 4 of 4 packets. *(Screenshot still to add.)*

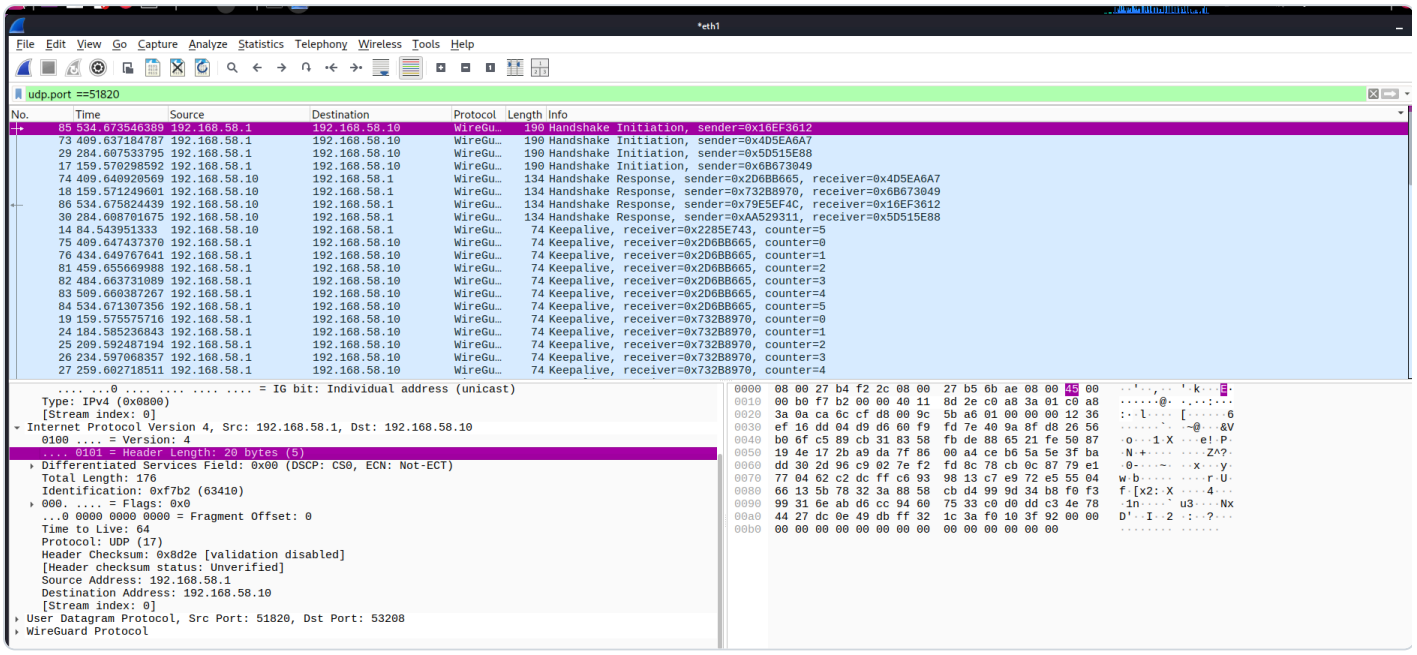
Wireshark: outside the tunnel versus inside it

I captured from two different places on Kali.

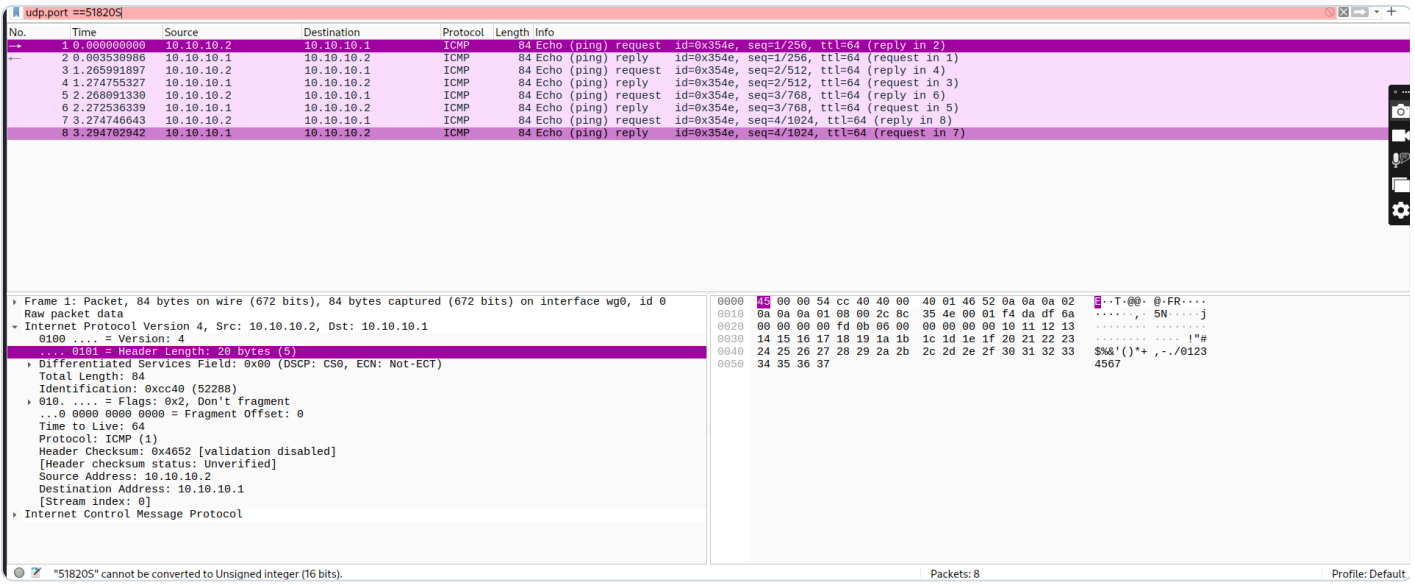
Outside the tunnel (interface eth1): I saw only UDP packets between 192.168.58.1 and 192.168.58.10 on port 51820: WireGuard Handshake Initiation, Handshake Response, Transport Data and small Keepalives. Anyone on the wire sees only this, and the contents are unreadable.



Filtering with `udp.port == 51820` lists the handshakes and keepalives:



Inside the tunnel (interface wg0): I saw the normal ICMP echo requests and replies between 10.10.10.2 and 10.10.10.1. This is the readable version, seen after decryption.



(In that screenshot the filter box still holds a typo, `udp.port ==51820S`, shown in red; the ICMP list is from the unfiltered capture.)

The difference between those two captures is the whole point of a VPN: the wire carries scrambled data, and only the two tunnel ends see the real packets.

What I would check first next time

1. Do the public keys match on both sides, character for character?
2. Did I click Apply Changes on pfSense?
3. Does `wg show` have a latest handshake?
4. Does `ip route get` say `dev wg0`?

Still to do

- Delete the stray second tunnel (`tun_wg1`).
- Change the default pfSense admin password (the warning banner is visible in several screenshots).
- Add a pre-shared key (it must match on both sides) as an extra layer.
- Revisit the interface assignment for `tun_wg0` when I write tunnel firewall rules.
- Add the `ip route get` and `ping 4/4` screenshots, and save the two capture files.